

waits until that lock is released before continuing. Every object has a lock that threads can acquire. Threads can acquire this through the lock construct. For the duration of the block, called a critical section, the thread holds the lock of the object releasing it at the end of the block. This allows for mutual exclusion over the counter variable: only one thread modifies or accesses counter at a time.

Note that locking leads to serialization of execution as threads wait for each other to release locks. There is a fine art to ensuring that just the right amount of locking is used: too little leads to race conditions and too much leads to serialized code which defeats the purpose of concurrency in the first place.

There are other issues that we won't go over due to time constraints. Two threads can end up waiting on locks that each other hold. This introduces a deadlock in your program.

Concurrency is a hot research area and developments and techniques are being constantly refined. Consider our discussion to be an introduction to the benefits and complexities of concurrency. 